

问题的背景：

seq2依赖于rnn是因为需要让模型感知词的相对位置和重要性，因此采用了位置编码并且完全基于注意力搭建

架构：- **编码器 (Encoder)**: 编码器由 $N = 6$ 个相同层的堆栈组成。每层有两个子层。第一个是多头自注意力机制，第二个是简单的逐位置全连接前馈网络。我们在两个子层周围采用残差连接，然后进行层归一化。也就是说，每个子层的输出是 $LayerNorm(x + Sublayer(x))$ ，其中 $Sublayer(x)$ 是子层本身实现的函数。为了促进这些残差连接，模型中的所有子层以及嵌入层产生的输出维度均为 $d_{model} = 512$ 。

- **解码器 (Decoder)**: 解码器也由 $N = 6$ 个相同层的堆栈组成。除了每个编码器层中的两个子层外，解码器还插入了第三个子层，该子层对编码器堆栈的输出执行多头注意力。与编码器类似，我们在每个子层周围采用残差连接，然后进行层归一化。我们还修改了解码器堆栈中的自注意力子层，以防止位置关注后续位置。这种掩码 (masking) 结合输出嵌入偏移一个位置的事实，确保了位置 i 的预测只能依赖于小于 i 位置的已知输出。

3.2 注意力 (Attention)

在实践中，我们同时在一组查询上计算注意力函数，并将其打包成矩阵 Q 。键和值也被打包成矩阵 K 和 V 。我们计算输出矩阵如下：

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

最常用的两种注意力函数是加法注意力 (additive attention) 和点积 (乘法) 注意力。点积注意力除了缩放因子 $1/\sqrt{d_k}$ 外，与我们的算法相同。加法注意力使用具有单个隐层的前馈网络计算兼容性函数。虽然两者在理论复杂性上相似，但在实践中，点积注意力更快且更节省空间，因为它可以使用高度优化的矩阵乘法代码来实现。

多头注意力允许模型同时关注来自不同位置的不同表示子空间的信息。对于单个注意力头，平均化会抑制这一点。

3.5 位置编码 (Positional Encoding)

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

训练：

5.1 训练数据和批处理 (Training Data and Batching)

我们在标准的 WMT 2014 英德数据集上进行训练，该数据集包含约 450 万个句子对。句子使用字节对编码 (BPE) 进行编码，具有约 37000 个标记的共享源-目标词汇表。对于英法翻译，我

们使用了显著更大的 WMT 2014 英法数据集，包含 3600 万个句子，并将标记拆分为 32000 个 word-piece 词汇表。

5.2 硬件和进度 (Hardware and Schedule)

我们在一台配备 8 个 NVIDIA P100 GPU 的机器上训练我们的模型。对于使用本文所述超参数的基础模型，每个训练步骤大约耗时 0.4 秒。基础模型训练了总共 100,000 步，耗时 12 小时。对于大模型（在表 3 的最后一行描述），步时为 1.0 秒。大模型训练了 300,000 步（3.5 天）。

5.3 优化器 (Optimizer)

我们使用 Adam 优化器，其中 $\beta_1 = 0.9, \beta_2 = 0.98$ 且 $\epsilon = 10^{-9}$ 。我们在训练过程中根据公式改变学习率：

$$lrate = d_{model}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$$

这对应于在第一个 *warmup_steps* 训练步中线性增加学习率，之后按步数的平方根倒数比例减少学习率。我们使用 *warmup_steps* = 4000。

5.4 正则化 (Regularization)

我们在训练期间采用三种类型的正则化：

- **残差丢弃 (Residual Dropout):** 我们将丢弃（dropout）应用于每个子层的输出，在其被加到子层输入并归一化之前。此外，我们将丢弃应用于编码器和解码器堆栈中嵌入和位置编码的总和。对于基础模型，我们使用率 $P_{drop} = 0.1$ 。
 - **标签平滑 (Label Smoothing):** 在训练期间，我们采用了值 $\epsilon_{ls} = 0.1$ 的标签平滑。这会影 响困惑度，因为模型学习变得更加不确定，但提高了准确性和 BLEU 分数
- 以下是同seq2论文对比显示的优越性

Seq2Seq vs. Transformer 性能与架构对比

维度	Seq2Seq (LSTM 架构)	Transformer (Attention 架构)	优越性说明	
翻译质量 (EN-FR BLEU)	34.81 (5个模型集成)	41.8 (单模型-Big版本)	显著提升：Transformer 单模型性能大幅超过 Seq2Seq 集成结果。	
训练耗时	10 天 (使用 8 块 GPU)	3.5 天 (Big版本) / 12 小时 (Base版本)	效率极高：在更短时间内达到更高的翻译精度。	
并行计算能力	低：计算具有序列依赖性，必须按步执行。	高：抛弃循环结构，所有位置可并行计算。	硬件利用率高：彻底解决了传统 RNN 无法大规模并行的瓶颈。	

维度	Seq2Seq (LSTM 架构)	Transformer (Attention 架构)	优越性说明	
长距离依赖路径	$O(n)$: 信号需遍历整个序列。	$O(1)$: 任何两个位置间操作数为常数。	建模能力更强 : 更易学习序列中相距较远的单词联系。	
长句处理技巧	必须依靠 源句子反转 来缩短初始依赖距离。	通过 自注意力机制 天然支持全局依赖。	结构更简洁 : 无需复杂的数据预处理技巧即可胜任长句翻译。	
模型深度	4 层 (4-layer LSTM)。	6 层 (6-layer Encoder/Decoder)。	可扩展性好 : 更深的结构配合残差连接提升了表达能力。	